Livrable L4 – Politique adaptative

Allocation adaptative des ressources de calcul et de recherche documentaire pour des systèmes LLM locaux, économes et fiables

Signaux, conception, réglage sur le jeu de développement et gel de la politique

Alae Lahbichi – Étudiant en 5e année IA/Data

a.lahbichi2004@gmail.com

Dépôt GitHub : AlaeLahbichi/Adaptive-Compute-and-RAG-for-Efficient-Reliable-Local-LLMs

18 octobre 2026 – politique figée : étiquette Git `politique-v1.0`

Résumé

Ce livrable présente la méthode proposée : une politique qui décide, question par question, de l'abstention, de la taille du modèle et de la profondeur de la recherche documentaire, à partir de deux signaux peu coûteux. Le premier est la *suffisance des preuves*, mesurée par le score BM25 du meilleur passage rapporté au score maximal possible. Le second est la *confiance* des modèles, mesurée par les probabilités des tokens générés. La politique est réglée uniquement sur le jeu de développement (180 questions), sous la contrainte d'être au moins 22 % plus rapide que la référence B (grand modèle + 3 passages). Une première conception en cascade (petit modèle d'abord) échoue : elle perd 14,6 points. La conception retenue s'abstient dès que la preuve est trop faible, sans générer, puis interroge le grand modèle. Elle obtient 0,552 de score global contre 0,541 pour B, avec 23 % de latence en moins. L'estimation par validation croisée, plus prudente, donne −2,1 points pour 22 % de latence en moins. Le gain repose surtout sur les questions non couvertes ; les questions simples perdent 14 points. La politique est figée avant l'évaluation finale (Partie V).

Table des matières

1 Rôle de cette partie 3

2 Signaux 3

3 La politique 3

4 Réglage 4

5 Démarche de conception 4

6 Politique figée 4

7 Résultats sur le jeu de développement 5

8 Ablations 6

9 Discussion 6

10 Limites 6

A Reproduire 7

1 Rôle de cette partie

La Partie III a montré qu'un routeur qui ne lit que la question ne satisfait pas le critère de succès, alors qu'un routeur parfait (oracle) ferait à la fois mieux et plus vite que B. La Partie IV cherche donc des *signaux plus informatifs* et les combine dans une politique interprétable, conformément au cahier des charges : « question simple → petit modèle ; preuves insuffisantes → nouvelle recherche ou abstention ; tâche difficile → modèle plus puissant ». Tout le réglage se fait sur le jeu de développement ; la politique est ensuite figée par l'étiquette Git `politique-v1.0`, avant toute exécution sur le jeu de test.

2 Signaux

Suffisance des preuves (gratuite). Pour une question q, la recherche BM25 renvoie le score S_1 du meilleur passage. On le rapporte au score maximal atteignable, $S_{\max}(q) = \sum_{t \in q} \mathrm{idf}(t)\,(k_1 + 1)$:

$$\mathrm{preuve}(q) = S_1 / S_{\max}(q) \in [0, 1].$$

Une valeur proche de 1 indique qu'un passage contient de nombreuses fois tous les mots importants de la question. Ce signal est un sous-produit de la recherche : il ne coûte rien.

Confiance des modèles (coût d'une génération). Pendant le décodage glouton, on relève la probabilité de chaque token produit. On en tire la moyenne géométrique (`conf_moy`) et le minimum (`conf_min`). Le décodage n'est pas modifié : les réponses sont identiques à celles de la Partie III, ce qui a été vérifié.

Pouvoir prédictif mesuré. Le tableau 1 indique dans quelle mesure chaque signal distingue les bonnes réponses des mauvaises (AUC : 0,5 = hasard, 1 = parfait).

Signal	**Ce qu'on cherche à prédire**	**AUC**
Confiance du petit modèle avec 3 passages (`conf_moy`)	Sa réponse est correcte	0,74
Confiance du grand modèle avec 3 passages (`conf_moy`)	Sa réponse est correcte	0,63
Confiance du grand modèle avec 6 passages (`conf_moy`)	Sa réponse est correcte	0,70
Confiance du petit modèle avec 3 passages (`conf_moy`)	La réponse de B est correcte	0,63
Preuve BM25	La réponse de B est correcte	0,55
Preuve BM25 (faible)	La question est non couverte	0,62

TABLE 1 – Pouvoir prédictif des signaux sur le jeu de développement. Les signaux sont informatifs, mais modérément : aucun ne dépasse 0,74.

3 La politique

Pour chaque question :

1. **Recherche BM25** de 6 passages et calcul de la preuve.
2. **Abstention précoce** : si preuve $< \tau_0$, réponse `UNANSWERABLE` immédiate, sans aucune génération.
3. **Cascade** (optionnelle) : le petit modèle répond avec 3 passages ; s'il ne s'abstient pas et que sa confiance dépasse τ_p, sa réponse est retenue.
4. **Grand modèle** sinon, avec 6 passages si preuve $< \tau_k$, 3 passages sinon.
5. **Abstention finale** si le grand modèle s'abstient ou si sa confiance est inférieure à τ_a.

Le coût de tous les signaux (recherche, génération du petit modèle quand elle a lieu) est compté dans la latence. Chaque décision est journalisée avec les étapes suivies, afin de pouvoir analyser les erreurs.

4 Réglage

Simulation exacte. Le décodage étant déterministe, la sortie de la politique pour une question est celle de l'étape où elle s'arrête. Les sorties de chaque étape (petit modèle à 3 passages, grand modèle à 3 et 6 passages) ont été calculées une fois sur le jeu de développement ; la politique est alors simulée exactement, latences comprises, pour tout réglage. Un contrôle en exécution réelle sur 6 questions de développement a confirmé l'enchaînement des étapes ; lors de l'évaluation finale, l'accord entre la politique exécutée et sa simulation est mesuré sur l'ensemble du jeu de test.

Grille. 246 510 réglages : type de confiance (moyenne ou minimum) pour chaque modèle, τ_p de 0 à 1 par pas de 0,025, τ_k et τ_0 sur des quantiles de la preuve, τ_a de 0,3 à 0,95, cascade activée ou non.

Objectif. Maximiser le score global sous la contrainte d'une latence moyenne au plus égale à 78 % de celle de B. Le critère de succès exige 80 % ; les 2 points de marge protègent contre la variabilité entre jeux de questions. En cas d'égalité, le réglage le plus rapide l'emporte.

Estimation honnête. Le réglage étant choisi sur les mêmes questions, son score est optimiste. On l'estime donc aussi par validation croisée en 5 plis : le réglage est choisi sur 4/5 des questions et appliqué au 1/5 restant.

5 Démarche de conception

Deux conceptions ont été essayées, toutes deux sur le jeu de développement uniquement.

1. **Cascade seule** (petit modèle, puis grand modèle si incertain, puis abstention si incertain). Meilleur réglage admissible : 0,395, soit −14,6 points par rapport à B, pour 24 % de latence en moins. Le petit modèle se trompe trop souvent, et sa confiance ne le révèle qu'en partie (AUC 0,74). Surtout, chaque escalade paie deux générations (0,37 s puis 0,92 s), ce qui limite le nombre d'escalades possibles sous la contrainte de latence.
2. **Ajout de l'abstention précoce sur preuve faible**, prévue par le cahier des charges (« preuves insuffisantes → abstention »). L'analyse des signaux montrait qu'une preuve faible est associée aux questions non couvertes. Le réglage, qui peut toujours activer la cascade, a retenu la version sans cascade.

Ces deux étapes sont rapportées pour la transparence : la politique finale est le résultat d'une exploration sur le jeu de développement, ce qui renforce l'importance de l'évaluation finale sur le jeu de test, non vu.

6 Politique figée

Paramètre	Valeur	Effet
Cascade (petit modèle d'abord)	non	Le petit modèle n'est jamais utilisé
Abstention précoce τ_0	preuve $< 0{,}265$	Environ 25 % des questions de développement, sans génération
Recherche complémentaire τ_k	preuve $< 0{,}286$	6 passages au lieu de 3 pour une petite partie des questions
Abstention finale τ_a	`conf_min` $< 0{,}40$	Abstention si un token de la réponse du grand modèle est très incertain

Table 2 – Paramètres figés (`configs/politique_v1.json`, étiquette `politique-v1.0`).

En pratique, la politique retenue se lit ainsi : *« Si les documents ne contiennent manifestement pas les mots de la question, s'abstenir sans rien générer. Sinon, interroger le grand modèle, avec plus de passages si la preuve est limite, et s'abstenir s'il hésite. »*

7 Résultats sur le jeu de développement

Méthode	Score	Écart à B	IC 95 %	Latence	Gain	Couv.	Risque
B – grand modèle, 3 passages	0,541	–	–	0,92 s	–	0,62	0,39
E – grand modèle, 6 passages	0,570	+2,9	[−1,1; +7,1]	1,63 s	−77 %	0,67	0,40
R1 – question seule	0,425	−11,6	[−18,5; −4,8]	0,74 s	20 %	0,58	0,57
R2 – Adaptive-RAG simplifié	0,522	−1,9	[−7,4; +3,3]	0,85 s	8 %	0,49	0,39
Politique (réglage figé)	**0,552**	**+1,1**	[−4,3; +6,4]	**0,71 s**	**23 %**	0,48	**0,34**
Politique (validation croisée)	0,520	−2,1	[−8,2; +3,8]	0,72 s	22 %	0,44	0,35
Oracle	*0,762*	+22,1	[+16,5; +27,8]	*0,40 s*	*56 %*	0,58	0,19

TABLE 3 – Jeu de développement (180 questions). Écarts en points de score, IC 95 % par bootstrap apparié. Gain : réduction de la latence moyenne par rapport à B.

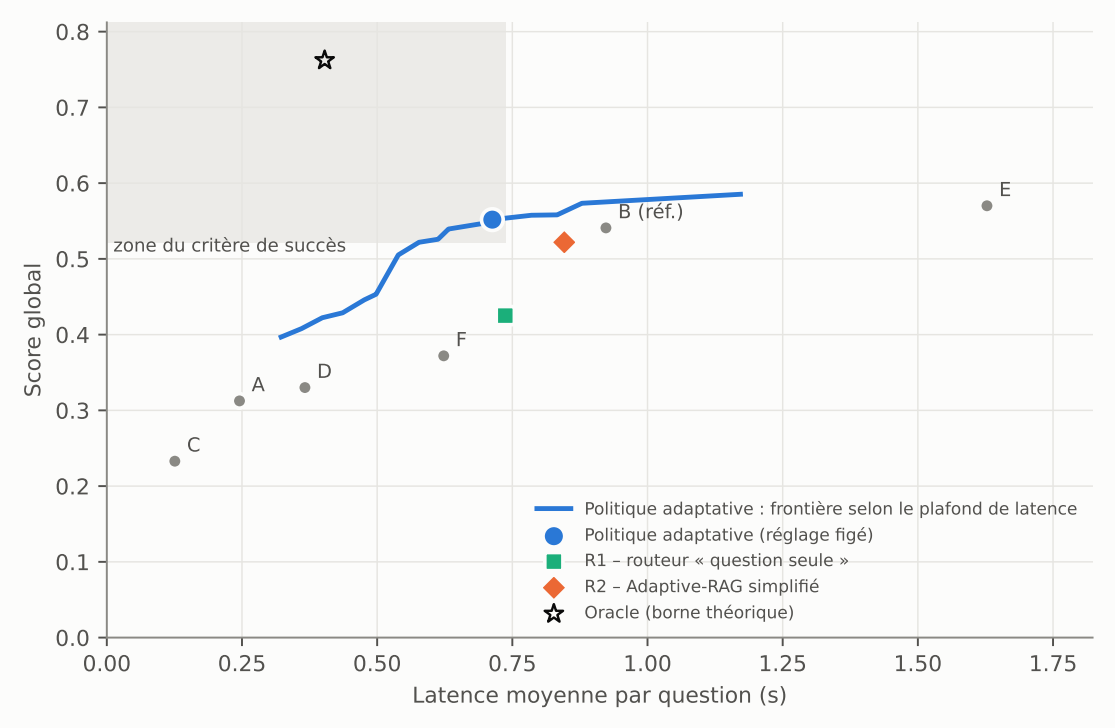


FIGURE 1 – Compromis qualité–coût sur le jeu de développement. La courbe bleue donne, pour chaque plafond de latence, le meilleur score atteignable par la politique ; le point bleu est le réglage figé.

Catégorie	B	Politique	Écart
Simple	0,678	0,538	−14,0
Complexe	0,362	0,317	−4,5
Non couverte	0,583	0,800	+21,7

TABLE 4 – Score par catégorie (60 questions chacune), jeu de développement.

Lecture. La politique égale B en score global tout en étant 23 % plus rapide, et elle se trompe moins souvent quand elle répond (risque 0,34 contre 0,39). Elle fait mieux que les deux routeurs de référence. Ce résultat a cependant un prix clair : elle répond à moins de questions (couverture 0,48 contre 0,62). Le gain vient des questions non couvertes, où l'abstention précoce est la bonne réponse, tandis que les questions simples perdent 14 points : la preuve BM25 classe à tort comme « faibles » certaines questions qui ont bien une réponse. Le critère secondaire de robustesse (aucune catégorie au-delà de 5 points de perte) n'est donc pas satisfait sur le jeu de développement.

8 Ablations

Chaque brique est retirée, puis la politique est réglée de nouveau sous la même contrainte de latence.

Variante	**Score**	**Écart à B**	**Latence**	**Couv.**
Politique complète (figée)	0,552	+1,1	0,71 s	0,48
Sans recherche adaptative (toujours 3 passages)	0,548	+0,7	0,68 s	0,47
Sans abstention finale	0,546	+0,5	0,71 s	0,49
Sans abstention précoce (meilleure cascade)	0,395	−14,6	0,71 s	0,77
Sans aucune abstention	0,389	−15,2	0,71 s	0,80

TABLE 5 – Ablations sur le jeu de développement. La variante « sans cascade » est identique à la politique complète, puisque le réglage retenu n'utilise pas la cascade.

Ce que montrent les ablations. L'*abstention précoce* porte presque tout le gain : sans elle, aucune variante ne s'approche de B sous la contrainte de latence. La recherche complémentaire et l'abstention finale apportent moins d'un point chacune, un écart non significatif sur 180 questions.

9 Discussion

— **La décision « faut-il répondre ? » compte plus que « quel modèle ? »**. Sous une contrainte de latence, il est plus rentable d'éviter de générer pour les questions sans réponse que de confier les questions faciles au petit modèle.

— **Le petit modèle n'est pas rentable ici** : il est nettement moins bon que le grand (0,33 contre 0,54 avec 3 passages), et sa confiance ne suffit pas à repérer ses erreurs. Avec un écart de taille de 2,7 seulement entre les deux modèles, une cascade ne paie pas.

— **Dépendance à la composition du benchmark** : un tiers des questions sont non couvertes, et le score global pèse les catégories à parts égales. Dans un usage où les questions sans réponse seraient rares, l'abstention précoce rapporterait moins et coûterait davantage en couverture. Ce point devra être discuté dans l'article.

10 Limites

— Réglage et exploration sur 180 questions seulement : l'écart entre le réglage figé (+1,1) et la validation croisée (−2,1) montre l'optimisme du premier.

— Deux conceptions ont été essayées ; d'autres signaux (accord entre réponses, reformulation de la question, recherche itérative) n'ont pas été explorés.

— Le signal de preuve est lexical : il pénalise les questions dont les mots diffèrent du texte source.

A Reproduire

```
HF_HUB_OFFLINE=1 python bench/run_strategies.py dev partie4   # étapes avec confiance
HF_HUB_OFFLINE=1 python bench/politique.py                    # réglage, VC, ablations
python bench/plot_partie4.py dev                              # figure
```